Project 2 Report: Extent-Based Software Page Table

Saroop

April 20, 2026

1 Overview

For project 2 I worked on modifying the Linux anonymous page fault path. The goal of the project was to group physically contiguous anonymous pages into extents, maintain those extents in a per-process red-black tree, and measure both the runtime overhead and the potential translation savings of extent-based mappings. The implementation was inserted into the anonymous page fault path so that a newly allocated private anonymous page is added to an extent only after the PTE is successfully installed. An extent stores the starting physical address, ending physical address, number of pages, a unique extent identifier, a linked list of physical pages in the extent, and an embedded red-black tree node. Each process maintains a red-black tree of extents inside `mm_struct`, keyed by extent start address. On each anonymous write fault, the handler looks for an existing covering extent or adjacent predecessor and successor extents. Depending on the result, the new page is inserted into an existing extent, merged between two extents, prepended to a following extent, or used to create a new extent. Thread safety is handled with a per-`mm` spinlock protecting the tree and extent metadata.

2 Implementation Summary

The implementation modified four kernel files:

- `include/linux/mm_types.h`: added `struct extent_page`, `struct extent`, and per-process extent metadata in `mm_struct`
- `kernel/fork.c`: initialized the extent tree, spinlock, and counters in `mm_init()`
- `mm/memory.c`: added extent lookup, insertion, merge logic, cleanup helpers, and the hook inside `do_anonymous_page()`
- `mm/mmap.c`: invoked extent cleanup during `exit_mmap()`

A key correctness detail was separating the zero-page read path from the real private anonymous page allocation path in `do_anonymous_page()`. The extent logic is only triggered when a new anonymous page is actually allocated and inserted into the page table. This avoids incorrectly counting read faults that are satisfied by the shared zero page.

3 Experimental Methodology

The benchmark was a custom user-space program, `pfbench.c`, that allocated a private anonymous `mmap` region and touched memory one 4 KiB page at a time. The benchmark supported both read-only and write-touch modes, as well as single-threaded and multithreaded execution. Transparent

huge pages were disabled for the benchmark range using `MADV_NOHUGEPAGE` so that the experiment would closely approximate one normal page fault per 4 KiB page touched.

For correctness experiments, I collected the number of extents printed at process exit. For performance analysis, I used `perf stat` with the events `page-faults`, `minor-faults`, `major-faults`, `task-clock`, `cycles`, `instructions`, `context-switches`, and `cpu-migrations`. The baseline kernel and the extent-enabled kernel were both measured on the same benchmark configuration. Because both kernels showed a slower first run, I report both the full five-run average and the steady-state average over runs 2 through 5.

4 Correctness Results

The extent output behaved as expected after filtering by the benchmark process identifier. A zero-work run (`p=0`) produced 11 extents, which captures the process startup baseline. A read-only run over 65,536 pages also produced 11 extents, showing that the zero-page path is not being counted as if it were allocating private anonymous pages. In contrast, a single-threaded write run over 65,536 pages produced 2,171 extents, and an 8-thread write run over the same region produced 2,577 extents. This shows that real anonymous page allocations are being grouped into extents and that the implementation remains stable under concurrent faults.

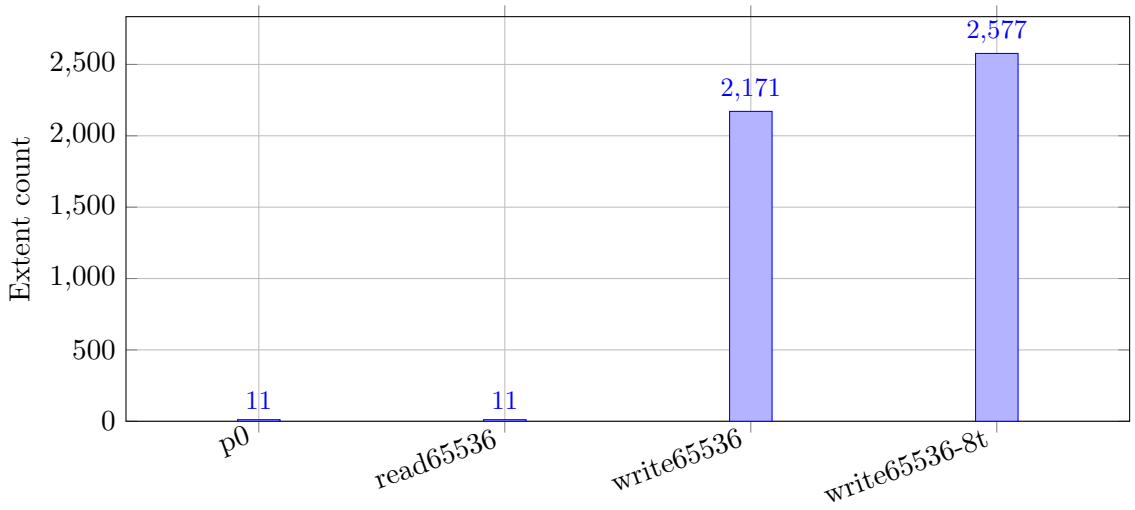


Figure 1: Extent counts observed for the benchmark under different workloads. The read-only run stays at the startup baseline, while write faults create and merge real anonymous extents.

5 Performance Analysis

Table 1 summarizes the single-threaded `perf stat` measurements for the 1 GiB write-touch benchmark (262,144 pages). The page-fault counts are essentially identical across the baseline and extent-enabled kernels, and all faults are minor faults with zero major faults. This is the expected pattern for first-touch anonymous memory. The main difference appears in runtime and instruction cost, which increase because the modified fault path must perform red-black tree lookup, possible extent merging, linked-list updates, and spinlock-protected metadata maintenance.

Table 1: Single-threaded performance summary for the 1 GiB write-touch benchmark.

Metric	Baseline avg. (runs 2–5)	Extent avg. (runs 2–5)	All-5 baseline avg.	All-5 extent avg.
Task-clock (ms)	472.33	501.845	501.296	537.972
Page faults	262,212.75	262,212.50	262,212.20	262,212.20
Minor faults	262,212.75	262,212.50	262,212.20	262,212.20
Major faults	0	0	0	0
Cycles	1,162,642,580.25	1,273,783,543.75	1,160,851,922.80	1,270,710,660.00
Instructions	821,815,476.75	1,007,084,963.25	821,830,627.40	1,007,024,391.00

The steady-state task-clock overhead is computed as

$$\frac{501.845 - 472.33}{472.33} \times 100 \approx 6.25\%.$$

If the first run is included, the average overhead becomes

$$\frac{537.972 - 501.296}{501.296} \times 100 \approx 7.32\%.$$

I treat the first run as a warm-up effect and use the runs 2 through 5 average as the main headline result.

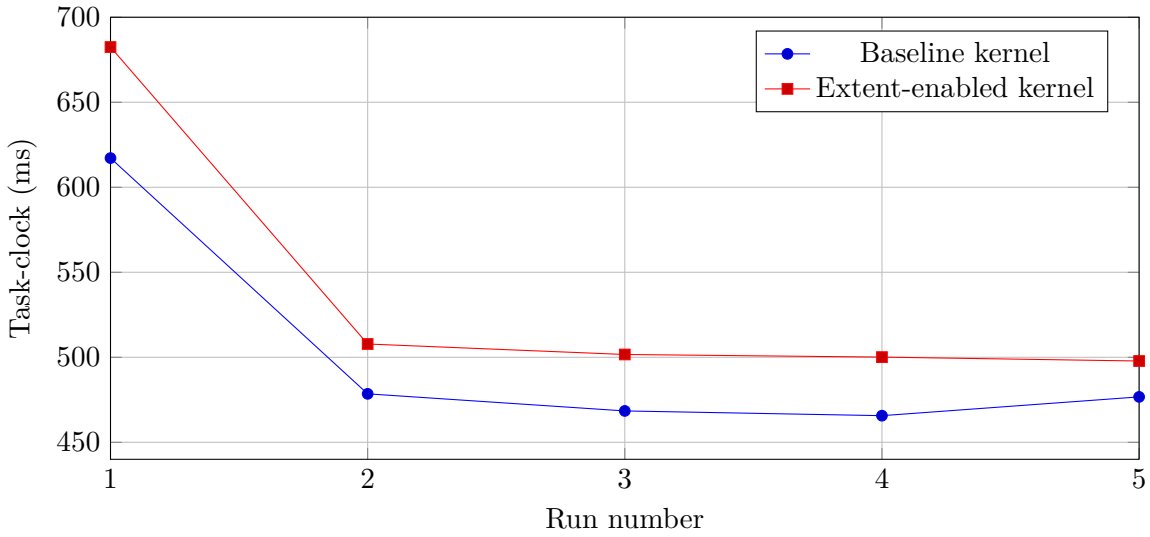


Figure 2: Task-clock across five repeated single-threaded runs of the 1 GiB write-touch benchmark. Both kernels show a slower first run, while steady-state behavior is more stable.

6 TLB Savings Estimate

To estimate the potential benefit of storing one TLB entry per extent instead of one entry per 4 KiB page, I used the single-threaded 65,536-page write benchmark. That run produced 2,171 extents. Under a page-based TLB, the process would require approximately 65,536 entries to represent the

region. Under an extent-based TLB, it would require only about 2,171 entries. The number of saved entries is therefore

$$65{,}536 - 2{,}171 = 63{,}365,$$

which corresponds to a reduction of

$$\frac{63{,}365}{65{,}536} \times 100 \approx 96.69\%.$$

If the process startup baseline of 11 extents is subtracted, the benchmark-specific extent count is approximately 2,160, which gives a nearly identical savings estimate of about 96.70%.

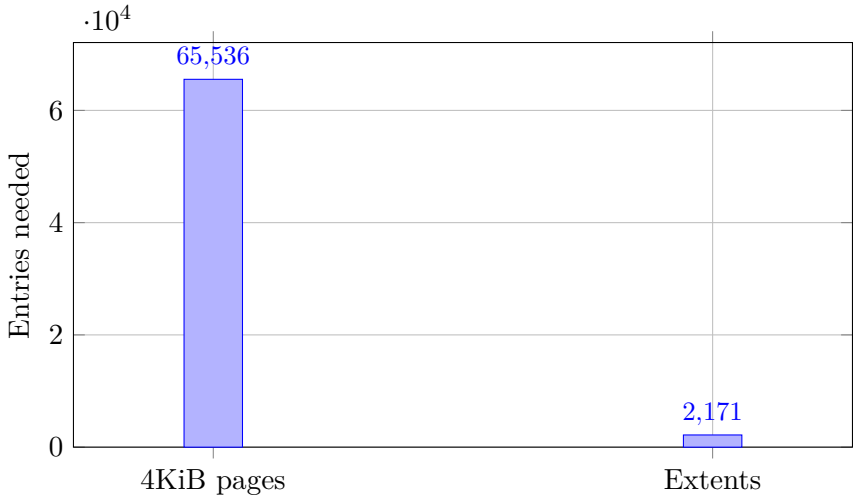


Figure 3: Estimated translation entries needed for the 65,536-page write benchmark. An extent-aware TLB would need dramatically fewer entries than a 4 KiB page-based design.

7 How I tested it:

I used the following compile command for the benchmark code pfbench.c

```
gcc -O2 -pthread -Wall -Wextra -o pfbench pfbench.c
```

The main correctness commands were:

```
./pfbench -p 0 -t 1 -m w -H 1
./pfbench -p 65536 -t 1 -m r -H 1
./pfbench -p 65536 -t 1 -m w -H 1
./pfbench -p 65536 -t 8 -m w -H 1
```

The main performance command was:

```
taskset -c 0 perf stat -x, \
  -e page-faults,minor-faults,major-faults,task-clock,cycles,instructions,\
context-switches,cpu-migrations -- ./pfbench -p 262144 -t 1 -m w -H 1
```